LLM Gateway Labs

Oct 28, 2026 · @jawahar gupta

Thirteen hands-on labs that build the LLM Gateway one feature at a time, each with prompts to paste into Claude Code and a checklist to confirm it works before you move on.

The labs at a glance

Each lab adds one working piece. Finish its checklist and commit before starting the next one. Labs 0 to 5 build the base; labs 6 to 9 add your four main features; labs 10 to 12 finish the project.

Lab	What you build	What you learn	Spec feature
0	Tools installed, empty repo, spec saved	Git, Claude Code basics, `CLAUDE.md`	Setup
1	FastAPI app with `/health` and settings	FastAPI, Uvicorn, Pydantic settings, `.env`	Base
2	Dockerfile and docker-compose	Images, containers, health checks	Docker
3	One provider + `POST /v1/chat/completions`	Async Python, schemas, calling an LLM API	1. Multi-LLM
4	4 providers, `models.yaml`, `/v1/models`, error types	Abstract classes, adapters, YAML config	1. Multi-LLM
5	Gateway API keys + test suite	Auth headers, pytest, mocking HTTP	Base
6	Traces and generations in Langfuse	Observability, tracing, structured logs	2. Observability
7	Complexity scorer + tier routing	Heuristics, routing rules	3. Routing
8	Retries, fallback chain, circuit breaker	Backoff, jitter, failure handling	3. Routing
9	Cost and savings tracking + `/v1/stats/costs`	Token pricing, `Decimal`, aggregation	4. Cost

Lab	What you build	What you learn	Spec feature
10	`prompts/` folder, `PromptLoader`, LLM classifier	Prompt versioning, Jinja2 templates	Prompt management
11	`stream: true` with Server-Sent Events	Async generators, SSE	1. Multi-LLM
12	Rate limiting, README, ADRs, final check	Production readiness, documentation	All

The full requirements live in the spec file `llm-gateway-v1.md`. These labs build the same thing in smaller steps.

How to work through a lab

Repeat the same five steps in every lab. The goal is to understand the code, not just to get it written.

1. **Start a fresh Claude Code session** in your project folder with `claude`, then paste the learning-mode prompt below.
2. **Paste the lab's "Understand" prompt first.** Read the explanation before any code is written.
3. **Paste the "Build" prompt.** Approve the plan, then let Claude write the code. Read every file it creates and ask about anything unclear.
4. **Run the checklist yourself.** Type the commands; don't only trust Claude saying it works.
5. **Commit** with the message given, so you can always go back to a working version.

Paste this at the start of every session:

```
I am a beginner learning backend development by building an LLM Gateway.
The full requirements are in docs/specs/llm-gateway-v1.md, but today we only
do the lab I describe next. Do not build features from later labs.

While you work:
- Explain each new concept in simple words before you use it.
- Show me the plan and wait for my OK before writing code.
- Keep the code simple and add short comments where something is not obvious.
- After writing code, tell me the exact commands to run and test it.
- At the end, ask me 2 short questions to check I understood.
```

Two habits that help a lot:

- **When you get an error**, paste the full error message and ask: "Explain what this error means, why it happened, and how to fix it."
- **When you don't understand code**, ask: "Explain `<file name>` line by line as if I am new to Python."

Lab 0: Set up your tools and project

Goal: a project folder under Git, with the spec saved and Claude Code ready. **You'll learn:** virtual environments, Git basics, how Claude Code reads project files.

Install these first (do this yourself, outside Claude Code):

- Python 3.12 and Git
- Docker Desktop (Windows/macOS) or Docker Engine (Linux)
- Claude Code: `curl -fsSL https://claude.ai/install.sh | bash` (macOS/Linux/WSL) or `irm https://claude.ai/install.ps1 | iex` (Windows PowerShell)
- An API key for at least one provider. If you have none, install Ollama to run a free local model.

Create the folder and start Claude Code:

```
mkdir llm-gateway && cd llm-gateway
git init
mkdir -p docs/specs
# copy llm-gateway-v1.md into docs/specs/
claude
```

Prompt: Understand

```
Read docs/specs/llm-gateway-v1.md. Explain to me in simple words:
1. What an LLM gateway is and why AI agents would use one.
2. What each of the 4 main features does, with a real-life example for each.
3. Which parts I will build first and why that order makes sense.
Do not write any code yet.
```

Prompt: Build

```
Set up the project skeleton only, no application code yet:
- Create a Python virtual environment in .venv and explain what it is.
- Create a .gitignore for Python (include .venv, .env, __pycache__).
- Create an empty README.md with the project name and a one-line description.
- Create requirements.txt (empty for now) and explain how we will use it.
Tell me the command to activate the virtual environment on my OS.
```

Check it works

- [ ] `python --version` shows 3.12 inside the activated `.venv`
- [ ] `docker --version` and `claude --version` both print a version
- [ ] `git status` shows your new files

Commit: `git add . && git commit -m "Lab 0: project setup"`

Lab 1: Your first FastAPI app

Goal: a running web server with `/health` and `/ready`, reading settings from `.env`. **You'll learn:** what an API endpoint is, FastAPI, Uvicorn, Pydantic settings, environment variables.

Prompt: Understand

```
Before we code, explain in simple words:
1. What FastAPI and Uvicorn are, and how they work together.
2. What an HTTP endpoint is, and the difference between GET and POST.
3. What environment variables and a .env file are, and why API keys must
never
   be written directly in code.
4. What Pydantic is and why it is useful for settings.
```

Prompt: Build

```
Lab 1: build the smallest working FastAPI app for the gateway.
- Folders: app/main.py, app/core/settings.py, app/api/health.py
- Settings with pydantic-settings, loaded from .env: APP_NAME, LOG_LEVEL,
  MODELS_CONFIG_PATH (not used yet).
- GET /health returns {"status": "ok"}.
- GET /ready returns {"status": "ready"} for now. Add a TODO saying it will
  check config and providers in later labs.
- Add fastapi, uvicorn and pydantic-settings to requirements.txt.
- Create .env.example with every setting and a comment for each.
Explain each file after you create it, then tell me how to run the server.
```

Check it works

- [ ] `pip install -r requirements.txt` finishes without errors
- [ ] `uvicorn app.main:app --reload` starts the server
- [ ] Opening `http://localhost:8000/health` shows `{"status":"ok"}`
- [ ] Opening `http://localhost:8000/docs` shows FastAPI's interactive API page

Then run `/init` inside Claude Code. It reads your new code and creates `CLAUDE.md`. Open the file, read it, and add one line: "Requirements are in docs/specs/llm-gateway-v1.md. Work only on the current lab."

Commit: `git add . && git commit -m "Lab 1: FastAPI app with health endpoints"`

Lab 2: Run it in Docker

Goal: the same app running inside a container, started with one command. **You'll learn:** images vs containers, Dockerfile, docker-compose, health checks, non-root users.

Prompt: Understand

```
Explain in simple words, with a real-life comparison:
1. What a Docker image and a container are, and how they differ.
2. What a Dockerfile and docker-compose.yml are each used for.
3. What a multi-stage build is and why it makes images smaller.
4. Why containers should not run as the root user.
```

Prompt: Build

```
Lab 2: containerize the app from Lab 1.
- Multi-stage Dockerfile on a slim Python 3.12 image.
- Run as a non-root user.
- HEALTHCHECK that calls /health.
- Start Uvicorn on port 8000; number of workers comes from UVICORN_WORKERS.
- docker-compose.yml with one service called gateway, reading .env,
  mapping port 8000.
- Add a .dockerignore.
Explain each line of the Dockerfile with a short comment.
```

Check it works

- [ ] `docker compose up --build` starts without errors
- [ ] `http://localhost:8000/health` returns `{"status":"ok"}`
- [ ] `docker ps` shows the container as `healthy` after about 30 seconds
- [ ] `docker compose down` stops it cleanly

Commit: `git add . && git commit -m "Lab 2: Docker setup"`

Lab 3: Your first LLM call through the gateway

Goal: `POST /v1/chat/completions` forwards a request to one provider and returns the answer in the OpenAI format. **You'll learn:** async/await, request and response schemas, calling an LLM API, what tokens are.

Use whichever provider you have a key for. If you have none, use Ollama on your machine.

Prompt: Understand

```
Explain in simple words:
1. What async and await mean in Python, and why a gateway needs them.
2. What the OpenAI Chat Completions request and response look like
   (messages, roles, choices, usage). Show me a small example of each.
3. What tokens are, and what input tokens and output tokens mean.
4. Why our gateway uses its own "unified" request/response models instead
   of passing provider formats straight through.
```

Prompt: Build

```
Lab 3: add the first LLM provider. Use <OpenAI | Anthropic | Ollama> only.
- app/schemas/: Pydantic models for a unified ChatRequest and ChatResponse
  (text, model, provider, input_tokens, output_tokens, finish_reason,
  latency_ms).
- app/providers/<name>.py: a class with async chat(request) -> ChatResponse
  that calls the provider and converts the result.
- POST /v1/chat/completions: accepts the OpenAI request format, calls the
  provider, returns the OpenAI response format.
- For now the model name comes from the request. No routing, retries,
  auth or streaming yet.
- Read the API key from .env. Never log it.
After building, give me a curl command to test it.
```

Check it works

- [ ] Your curl command returns a real answer from the model
- [ ] The response has a `usage` section with token counts
- [ ] Removing the API key from `.env` gives a clear error, not a crash

Ask yourself: where in the code is the provider's response turned into the gateway's format?

Commit: `git add . && git commit -m "Lab 3: first provider and chat endpoint"`

Lab 4: Many providers, one interface

Goal: OpenAI, Anthropic, Gemini and Ollama all work behind the same endpoint, chosen by model name from `config/models.yaml`. **You'll learn:** abstract base classes, the adapter pattern, YAML config, turning many error types into a few.

Prompt: Understand

```
Explain in simple words with a real-life example:
1. What an abstract base class is in Python.
2. What the adapter pattern is, and why it lets us add providers without
   changing the rest of the gateway.
3. How the request formats of OpenAI and Anthropic differ (for example,
   where the system prompt goes).
4. Why we turn every provider's errors into our own few error types.
```

Prompt: Build

```
Lab 4: support multiple providers. Follow "Feature 1" and the
config/models.yaml format in the spec.
- app/providers/base.py: abstract BaseProvider with async chat().
  (stream_chat comes in Lab 11, leave it out.)
- Move the Lab 3 provider onto BaseProvider, then add adapters for the
  other providers: OpenAI, Anthropic, Gemini, Ollama.
- config/models.yaml with providers and models. Use placeholder model IDs
  and prices marked "# TODO: verify". Validate it with Pydantic at startup.
- A provider registry that picks the adapter from the model name.
- Error types: RateLimitError, TimeoutError, AuthError,
  ProviderUnavailableError, BadRequestError. Each adapter converts its
  provider's errors into these.
- GET /v1/models lists configured models.
- Update /ready to check that config loaded and at least one provider has
  credentials.
Show me how adding a fifth provider would work.
```

Check it works

- [ ] Changing only `model` in your curl request switches between providers you have keys for
- [ ] `/v1/models` lists every model in the YAML file
- [ ] A typo in `models.yaml` stops startup with a clear message
- [ ] An unknown model name returns an error, not a crash

Ask yourself: which files would change if you added a new provider tomorrow?

Commit: `git add . && git commit -m "Lab 4: multi-provider support"`

Lab 5: Lock it down and add tests

Goal: only agents with a gateway key can call the API, errors use one standard format, and a test suite runs without real API calls. **You'll learn:** Bearer tokens, FastAPI dependencies, pytest, mocking HTTP calls, why tests must not call real APIs.

Prompt: Understand

```
Explain in simple words:
1. What "Authorization: Bearer <key>" means and how a server checks it.
2. What a FastAPI dependency is.
3. What unit tests and integration tests are, with one example of each
   for our gateway.
4. What mocking is, and why our tests must never call a real LLM API.
```

Prompt: Build

```
Lab 5: add auth, the standard error format, and tests.
- Auth: agents send Authorization: Bearer <gateway-key>. Valid keys come
  from GATEWAY_API_KEYS in .env. /health and /ready stay public.
- Errors: use the OpenAI error shape and the status-code table from the
  "Error format" section of the spec.
- Add X-Request-Id: use the incoming header or generate one, and return it
  in every response.
- Tests with pytest, pytest-asyncio and respx:
  - unit tests for each provider's request/response conversion
  - integration tests for /v1/chat/completions: success, missing key (401),
    wrong key (401), unknown model (404)
- Put sample provider responses in tests/fixtures/.
- Add ruff and fix any lint problems.
Teach me how to run one test, all tests, and see coverage.
```

Check it works

- [ ] A request without a key returns 401 in the error format from the spec
- [ ] `pytest` passes with your internet turned off
- [ ] `ruff check .` reports no problems
- [ ] Every response has an `X-Request-Id` header

Ask yourself: if a test fails, how do you tell whether the bug is in the test or in the code?

Commit: `git add . && git commit -m "Lab 5: auth, errors and tests"`

Lab 6: See every call in Langfuse

Goal: every request shows up in Langfuse with its model, tokens, latency and agent ID, and the gateway writes structured JSON logs. **You'll learn:** observability, traces vs generations, metadata, why monitoring must never break the main request.

Before you start, create a free account on Langfuse Cloud (or self-host it later in Lab 12) and copy your public key, secret key and host into `.env`.

Prompt: Understand

```
Explain in simple words:
1. What observability means for an LLM application, and what problems
   it helps us find.
2. In Langfuse, what a trace and a generation are, and how they relate.
   Use one request to our gateway as the example.
3. What structured (JSON) logs are and why they beat print statements.
4. Why the gateway must keep working even if Langfuse is down.
```

Prompt: Build

```
Lab 6: add observability. Follow "Feature 2" in the spec.
- Use the current Langfuse Python SDK. Check its docs for the up-to-date
  API before writing code, and tell me which version you used.
- Each request = one trace. Each provider call = one generation with model,
  provider, input, output, token counts, latency, status and error message.
- Read X-Agent-Id, X-Session-Id and X-User-Id headers and attach them to
  the trace, along with the request ID.
- If Langfuse keys are missing or Langfuse fails, log a warning and carry on.
- Flush Langfuse events when the app shuts down.
- Structured JSON logs for every request. Do not log message content by
  default (LOG_PROMPTS=false).
- Tests: mock Langfuse; check a request still succeeds when Langfuse errors.
```

Check it works

- [ ] After one curl request, a trace appears in your Langfuse dashboard
- [ ] The trace shows tokens, latency and your `X-Agent-Id`
- [ ] With wrong Langfuse keys, requests still succeed and a warning is logged
- [ ] `pytest` still passes

Ask yourself: what would you look for in Langfuse if users said the gateway felt slow?

Commit: `git add . && git commit -m "Lab 6: Langfuse observability"`

Lab 7: Pick the right model for each prompt

Goal: with `model: "auto"`, short easy prompts go to a cheap model and long, hard prompts go to a strong one. **You'll learn:** heuristics, scoring, thresholds, config-driven rules, interfaces you can swap later.

Prompt: Understand

```
Explain in simple words:
1. Why sending every prompt to the most powerful model wastes money.
2. What a heuristic is, and which simple signals can hint that a prompt
   is hard (length, code blocks, words like "analyze" or "step by step").
3. How a score from 0 to 1 plus thresholds turns into simple, medium or
   complex.
4. Show me 3 example prompts and the tier you would expect for each.
```

Prompt: Build

```
Lab 7: add complexity-based routing. Follow "Feature 3" in the spec, but
only the scorer and tiers. No retries or fallback yet (that is Lab 8).
- app/routing/complexity.py: a ComplexityScorer interface and a
  HeuristicScorer that returns a score from 0 to 1 and the reasons for it.
- Tiers and thresholds in config/models.yaml under "routing".
- app/routing/router.py: if model is "auto", score the prompt, pick the
  tier and use the first model in that tier. A real model name skips
  routing.
- X-Route-Tier request header forces a tier.
- Response headers: X-Model-Used, X-Route-Tier.
- Send the score, tier and reason to Langfuse as trace metadata.
- Unit tests with at least 6 example prompts across all three tiers.
```

Check it works

- [ ] "What is 2+2?" with `model: "auto"` returns `X-Route-Tier: simple`
- [ ] A long prompt asking to analyze code returns `X-Route-Tier: complex`
- [ ] `X-Route-Tier: medium` forces the medium tier
- [ ] Langfuse shows the complexity score on the trace

Try this: change the thresholds in `models.yaml` and see how the same prompt changes tier, without touching code.

Commit: `git add . && git commit -m "Lab 7: complexity routing"`

Lab 8: Survive provider failures

Goal: when a provider is slow, rate-limited or down, the gateway retries, then falls back to the next model, and stops calling a provider that keeps failing. **You'll learn:** retryable vs non-retryable errors, exponential backoff, jitter, fallback chains, the circuit breaker pattern.

Prompt: Understand

```
Explain in simple words with real-life comparisons:
1. Which errors are worth retrying (rate limit, timeout, 5xx) and which are
   not (bad request, wrong API key), and why.
2. What exponential backoff and jitter are. Show the wait times for 4
   retries with base 0.5s.
3. What a fallback chain is, and why it should include different providers.
4. What a circuit breaker is and its three states: closed, open, half-open.
```

Prompt: Build

```
Lab 8: add retries, fallback and a circuit breaker. Follow "Feature 3" in
the spec.
- app/routing/retry.py: exponential backoff with jitter for retryable
  errors only. Respect Retry-After. Settings from the "retry" section of
  models.yaml.
- Fallback: when retries for a model run out, or the provider has an auth
  or availability error, try the next model in the tier. Never fall back on
  BadRequestError.
- app/routing/circuit_breaker.py: per provider, open after N failures,
  wait cooldown_s, then allow one trial request (half-open).
- X-No-Fallback: true header disables fallback.
- Response header X-Fallback-Used. When every model fails, return 503 in the
  standard error format.
- Each attempt is its own generation in the same Langfuse trace.
- Tests: retry then success, fallback to the second provider, all fail
  (503), BadRequest does not fall back, circuit breaker state changes.
  Use fake sleep in tests so they run fast.
```

Check it works

- [ ] Put a wrong API key for your first provider (you need a second working provider, such as Ollama): the request still succeeds and `X-Fallback-Used: true`
- [ ] Langfuse shows the failed attempt and the successful one in a single trace
- [ ] With every key wrong, you get a clean 503, not a crash
- [ ] All tests pass in a few seconds

Ask yourself: why is retrying a "wrong API key" error a waste of time?

Commit: `git add . && git commit -m "Lab 8: retries, fallback, circuit breaker"`

Lab 9: Measure cost and savings

Goal: every response shows what it cost, what the baseline model would have cost, and how much routing saved; `/v1/stats/costs` shows the totals. **You'll learn:** how LLM pricing works, why money uses `Decimal`, running totals, grouping data.

Before you start, open each provider's pricing page and fill in the real prices in `models.yaml` (price per 1 million input tokens and per 1 million output tokens).

Prompt: Understand

```
Explain in simple words:
1. How LLM providers charge (input vs output tokens, price per 1M tokens).
   Work through one example by hand: 1,200 input tokens and 300 output
   tokens at made-up prices.
2. What a baseline model is and how we calculate savings against it.
3. Why we use Decimal instead of float for money in Python.
4. Why in-memory totals reset on restart and differ per worker, and how
   Redis fixes that.
```

Prompt: Build

```
Lab 9: add cost tracking. Follow "Feature 4" in the spec.
- app/cost/pricing.py: cost from token counts and models.yaml prices,
  using Decimal.
- For every request compute actual_cost, baseline_cost (using
  baseline_model from the config), savings and savings_pct.
- Add the "gateway" object from the spec's API contract to each response.
- Send the cost numbers to Langfuse on the trace.
- app/cost/tracker.py: running totals in memory, with a Redis option when
  COST_STORE=redis.
- GET /v1/stats/costs with totals and breakdown by model, tier and agent ID;
  optional query params since and agent_id.
- Unit tests that compare against hand-calculated numbers.
```

Check it works

- [ ] One response's `gateway.cost` matches your own calculation from its token counts
- [ ] After 5 mixed requests, `/v1/stats/costs` shows totals and a breakdown by tier
- [ ] `?agent_id=agent-1` shows only that agent's requests
- [ ] Langfuse shows cost on each trace

Ask yourself: if 80% of requests route to the simple tier, roughly what savings percent would you expect?

Commit: `git add . && git commit -m "Lab 9: cost tracking"`

Lab 10: Manage prompts like code

Goal: prompts the gateway sends to LLMs live in versioned YAML files under `prompts/`, and an optional LLM-based classifier uses one of them. **You'll learn:** separating prompts from code, versioning, Jinja2 templates, Langfuse Prompt Management, safe fallbacks.

Prompt: Understand

```
Explain in simple words:
1. Why prompts should not be hard-coded inside Python strings.
2. How versioning prompts (v1, v2) helps when a new prompt performs worse.
3. What a Jinja2 template is and what StrictUndefined protects us from.
4. The pros and cons of keeping prompts in files vs in Langfuse Prompt
   Management.
5. Compare our heuristic scorer with an LLM-based classifier: cost, speed,
   accuracy.
```

Prompt: Build

```
Lab 10: add runtime prompt management. Follow "Runtime prompt management"
in the spec.
- prompts/complexity_classifier/v1.yaml in the format from the spec,
  plus prompts/README.md explaining how to add or change a prompt.
- app/prompts/loader.py: PromptLoader.get(name, version), Pydantic
  validation at startup, Jinja2 rendering with StrictUndefined, caching.
- Optional Langfuse source when PROMPT_SOURCE=langfuse, with fallback to the
  local file if Langfuse is unreachable.
- LLMScorer that uses the classifier prompt, enabled with
  COMPLEXITY_SCORER=llm. On any error it falls back to HeuristicScorer.
- Record prompt_name and prompt_version on the Langfuse generation.
- Tests: load latest and a specific version, missing variable raises an
  error, invalid file fails startup, Langfuse failure falls back to local.
```

Check it works

- [ ] With `COMPLEXITY_SCORER=llm`, Langfuse shows a classifier call with its prompt name and version
- [ ] Creating `v2.yaml` makes the loader pick it up as the latest
- [ ] Removing a `{{ variable }}` the code needs stops startup with a clear error
- [ ] With the classifier's provider down, routing still works using the heuristic scorer

Ask yourself: when would the extra cost of an LLM classifier be worth it?

Commit: `git add . && git commit -m "Lab 10: prompt management"`

Lab 11: Stream answers word by word

Goal: with `stream: true`, the answer arrives in small pieces as the model writes it, in the OpenAI streaming format. **You'll learn:** Server-Sent Events (SSE), async generators, `StreamingResponse`, why fallback is harder once streaming has started.

Prompt: Understand

```
Explain in simple words:
1. Why streaming makes chat apps feel faster even when the total time
   is the same.
2. What Server-Sent Events are, and what a "data: ..." line and
   "data: [DONE]" mean.
3. What an async generator is in Python, with a tiny example.
4. Why the gateway can fall back to another model before the first token,
   but not after it has started sending text.
```

Prompt: Build

```
Lab 11: add streaming. Follow the streaming rules in the spec's API
contract.
- Add async stream_chat() to BaseProvider and implement it in every
  adapter.
- /v1/chat/completions with stream: true returns SSE chunks in the OpenAI
  format, ending with data: [DONE].
- The last chunk before [DONE] includes usage and the "gateway" object.
- Fallback is allowed only before the first token. After that, a provider
  error ends the stream with an error event.
- Langfuse and cost tracking must still record the full output and tokens.
- Tests for a normal stream, fallback before the first token, and an error
  mid-stream.
```

Check it works

- [ ] `curl -N` with `"stream": true` prints the answer piece by piece
- [ ] The stream ends with `data: [DONE]`
- [ ] Langfuse shows the full answer and token counts for a streamed request
- [ ] `/v1/stats/costs` counts streamed requests

Commit: `git add . && git commit -m "Lab 11: streaming"`

Lab 12: Make it production-ready

Goal: rate limiting, full documentation, a complete Docker setup with optional Redis, Ollama and self-hosted Langfuse, and every acceptance criterion in the spec passing.
You'll learn: rate limiting, architecture decision records (ADRs), reusable Claude Code commands, reviewing a project against its spec.

Prompt: Understand

```
Explain in simple words:
1. What rate limiting is and why a gateway needs it per API key.
2. What an ADR (architecture decision record) is, with a short example.
3. What docker-compose profiles are and when to use them.
4. What "acceptance criteria" are and how we check each one.
```

Prompt: Build

```
Lab 12: finish the project.
- Rate limiting per gateway key (RATE_LIMIT_PER_MINUTE), returning 429 in
  the standard error format. Request body size limit.
- docker-compose profiles for redis, ollama and a self-hosted Langfuse
  stack (follow Langfuse's official self-hosting setup).
- docs/architecture.md with a Mermaid diagram of the request flow.
- docs/adr/: at least four ADRs, as listed in the spec.
- .claude/commands/: add-provider.md, add-model.md, review.md.
- README.md as described in the spec, including how to point the OpenAI
  Python SDK at the gateway by changing base_url.
- Update CLAUDE.md so it matches the finished project.
```

Prompt: Final review

```
Go through every item in the "Acceptance criteria" section of
docs/specs/llm-gateway-v1.md. For each one, tell me: pass or fail, how you
checked it, and what to fix if it fails. Then list remaining TODOs and any
assumptions you made. Do not fix anything yet; I want to review first.
```

Check it works

- [ ] More requests than the limit within a minute return 429
- [ ] The OpenAI Python SDK works against the gateway by changing only `base_url` and `api_key`
- [ ] `docker compose --profile redis up` runs the gateway with Redis cost totals
- [ ] Every acceptance criterion in the spec is marked pass
- [ ] Coverage targets are met and `ruff check .` is clean

Commit: `git add . && git commit -m "Lab 12: production readiness"`, then tag it: `git tag v1.0`

After Lab 12, good next steps are deploying to a cloud server and adding a small dashboard that reads `/v1/stats/costs`.